ការអនុវត្តន៍ជាមួយ ASP.NET API

ឯកសារណែនាំនេះត្រូវបានរចនាឡើងសម្រាប់អ្នកអភិវឌ្ឍន៍កម្មវិធី និងវិស្វករ ASP.NET ដែលមានជំនាញមូលដ្ឋានលើ Web API។ វានឹងណែនាំអ្នកតាមរយៈគំនិតស្នូលនៃ REST, គោលការណ៍រចនា API, ការអនុវត្ត CRUD Operations, និងល្អបំផុតសម្រាប់ការបង្កើត ASP.NET Core Web API ប្រកបដោយគុណភាព។

១. សេចក្តីសង្ខេបខ្លីអំពី REST

REST ឬ Representational State Transfer គឺជារចនាប័ទ្មស្ថាបត្យកម្មមួយសម្រាប់ Web API ដែលត្រូវបានបង្កើតឡើងដោយ Roy Fielding ក្នុងការស្រាវជ្រាវបណ្តុះបណ្តាល Ph.D របស់គាត់។ វាបានក្លាយជាស្តង់ដារដ៏ពេញនិយមមួយក្នុងការបង្កើតសេវាកម្ម Web ដោយសារភាពសាមញ្ញ និងភាពបត់បែនរបស់វា។

REST ជារចនាប័ទ្ម មិនមែនច្បាប់

គ្រាន់តែចងចាំថា REST គឺជារចនាប័ទ្ម ប៉ុន្តែមិនមែនជាច្បាប់ទេ។ នៅពេលអ្នកបង្កើត Web API មួយ អ្នកមិនចាំបាច់ត្រូវតែធ្វើតាមរចនាប័ទ្ម REST ទេ។ ទោះជាយ៉ាងណា ការធ្វើតាម REST guidelines ជួយឱ្យ API របស់អ្នកមានភាពស្ថិតស្ថេរ ងាយស្រួលប្រើ និងអាចថែទាំបានក្នុងរយៈពេលវែង។

ប្រវត្តិ REST

Roy Fielding បានបង្ហាញ REST ជាលើកដំបូងក្នុងឆ្នាំ ២០០០ ក្នុងការស្រាវជ្រាវ Ph.D របស់គាត់ ដែលមានចំណងជើងថា "Architectural Styles and the Design of Network-based Software Architectures"។ ចាប់តាំងពីពេលនោះមក REST បានក្លាយជាមូលដ្ឋានសំខាន់នៃ Web API ទំនើប។

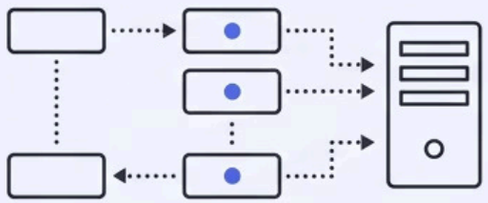
២. The Constraints of REST

REST មានលក្ខខណ្ឌ ៦ ដើម្បីឱ្យ API ត្រូវបានចាត់ទុកថា RESTful។ លក្ខខណ្ឌទាំងនេះជួយធ្វើឱ្យ API មានភាពស្ថិតស្ថេរ អាចពង្រីកបាន និងងាយស្រួលប្រើ។ ការយល់ដឹងអំពីលក្ខខណ្ឌទាំងនេះជួយអ្នកអភិវឌ្ឍន៍ក្នុងការបង្កើត API ដែលប្រកាន់ខ្ជាប់ REST Principles។

1) កូនសេវា-មេយ៉ា

ការបំបែក UI និងការផ្ទុកទិន្នន័យ

2) ឥតស្ថានភាព

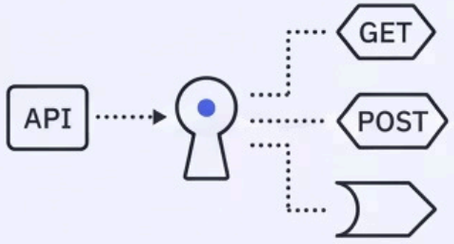


សំណើមានព័ត៌មានគ្រប់គ្រាន់

3) ដែលអាចរក្សាទុកក្នុងឃ្លាំង

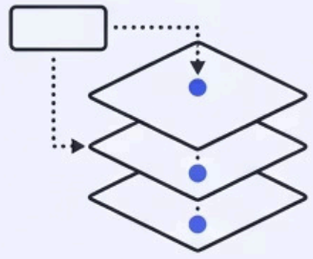
ចម្លើយអាចត្រូវបានរក្សាទុក

4) ចំណុចប្រទាក់ឯកសណ្ឋាន



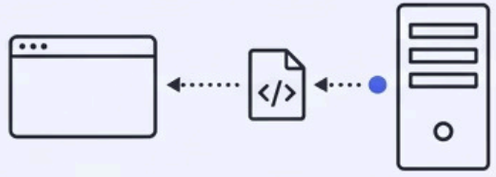
ចំណុចប្រទាក់ API ស៊ីសង្វាក់គ្នា

5) ប្រព័ន្ធស្រទាប់



ស្រទាប់តាមឋានានុក្រម

6) កូដតាមតម្រូវការ



មេយ៉ាធ្វើកូដដែលចោលប្រតិបត្តិបាន

២.១. អំពី Client-Server

គោលការណ៍ Client - Server គឺជាគំនិតស្នូលរបស់ REST ដែលបំបែកការទទួលខុសត្រូវ (Separation of Concerns) រវាង User Interface (Client) និង Data Storage (Server)។ ការបំបែកនេះជួយឱ្យ Client និង Server អាចអភិវឌ្ឍបានឯករាជ្យពីគ្នា ដោយគ្មានការប៉ះពាល់ទៅលើគ្នា។

Client (អ្នកស្នើសុំ)

Client ទទួលខុសត្រូវចំពោះការបង្ហាញ UI, ការគ្រប់គ្រង User Interaction, និងការធ្វើ Request ទៅ Server។ Client មិនចាំបាច់ដឹងពីរបៀបដែល Server ផ្ទុករក្សាទុកទិន្នន័យទេ។

Server (អ្នកឆ្លើយតប)

Server ទទួលខុសត្រូវចំពោះការផ្ទុក Business Logic, ការគ្រប់គ្រង Database, និងការឆ្លើយតបទៅ Client។ Server មិនចាំបាច់ដឹងពីរបៀបដែល Client បង្ហាញទិន្នន័យទេ ដែលធ្វើឱ្យប្រព័ន្ធមានភាពបត់បែន និងអាចថែទាំបានកាន់តែងាយ។

២.២. Stateless (The Constraints of REST)

Stateless មានន័យថា Server មិនរក្សា Session State ណាមួយអំពី Client នៅចន្លោះ Requests ទេ។ Request នីមួយៗពី Client ត្រូវតែមានព័ត៌មានគ្រប់គ្រាន់ ដើម្បីឱ្យ Server អាចយល់ និងដំណើរការ Request នោះបានដោយខ្លួនឯង ដោយមិនពឹងផ្អែកលើ Context ពី Request មុន។

អត្ថប្រយោជន៍

Visibility, Reliability, Scalability ត្រូវបានធ្វើអោយប្រសើរ ពីព្រោះ Server អាចដំណើរការ Request ណាមួយដោយឯករាជ្យ។

ការណ៍ប្រយ័ត្ន

ទិន្នន័យ Session ត្រូវតែរក្សាទុកនៅ Client Side ហើយបញ្ជូនរួមជាមួយ Request នីមួយៗ ដូចជា Authentication Token ជាដើម។

ឧទាហរណ៍

JWT Token ត្រូវបញ្ជូននៅក្នុង Authorization Header ជារៀងរាល់ Request ដើម្បីបញ្ជាក់អត្តសញ្ញាណ User។

២.៣. Cacheable (The Constraints of REST)

លក្ខខណ្ឌ Cacheable មានន័យថា Responses ពី Server ត្រូវតែកំណត់ (Labeled) ថាអាច Cache បាន ឬមិនអាច Cache បានដោយ Explicit ឬ Implicit។ ប្រសិនបើ Response អាច Cache បាន Client ឬ Proxy Cache អាចប្រើ Response នោះជា Response នៃ Request ស្រដៀងគ្នា នៅពេលក្រោយ ដូច្នេះការកាត់បន្ថយ Latency និង Server Load។

ធ្វើអោយ Performance ប្រសើរ

Caching ជួយកាត់បន្ថយ Latency ដោយបម្រើ Cached Responses ភ្លាមៗ ដោយមិនចាំបាច់ query ទៅ Database ម្តងទៀត ធ្វើឱ្យ Response Time លឿនជាង។

ការ Cache Headers

ប្រើ HTTP Headers ដូចជា Cache-Control, ETag, Last-Modified ដើម្បីគ្រប់គ្រង Cache Behavior ធ្វើឱ្យ Client ដឹងថាតើ Response ត្រូវ Cache ប៉ុន្មាន ឬ Refresh នៅពេលណា។

Stale Data

ការ Cache ខុស Cache ខុស អាចធ្វើឱ្យ Client ទទួល Stale Data ។ ត្រូវរាង Cache Duration ឱ្យត្រឹមត្រូវ ដើម្បីធ្វើបុប្បា Balance ល្អរវាង Performance និងTimeliness នៃ Data។

២.៤. អំពី Uniform Interface (The Constraints of REST)

Uniform Interface គឺជាលក្ខខណ្ឌស្នូលដ៏សំខាន់បំផុតរបស់ REST ដែលកំណត់ Interface រួមមួយសម្រាប់ Component នៃ REST System ទំនាក់ទំនងជាមួយគ្នា។ វារួមមាន ៤ គំនិតរង ដែលណែនាំរបៀបដែល Client និង Server ត្រូវ Interact គ្នា។

២.៥. អំពី Layered System (The Constraints of REST)

Layered System អនុញ្ញាតឱ្យ Architecture ប្រើ Intermediate Layers ដូចជា Proxy, Load Balancer, Gateway, Security Layer ជាដើម រវាង Client និង Server ។ Client មិនចាំបាច់ដឹងថាគាត់ communication ដោយផ្ទាល់ជាមួយ Server ចុងក្រោយ ឬ Intermediary ទេ ដែលធ្វើឱ្យ Architecture មានភាពបត់បែន។

1

Client

ធ្វើ Request ដោយមិនដឹងថានឹងឆ្លងកាត់ Layer ប៉ុន្មាន

2

Gateway / Proxy

Security, Load Balancing, Authentication, Rate Limiting

3

Server

ដំណើរការ Business Logic ហើយ Return Response

២.៦. អំពី Code on Demand (Optional)

Code on Demand គឺជាលក្ខខណ្ឌ Optional តែមួយគត់ក្នុងចំណោម ៦ លក្ខខណ្ឌរបស់ REST។ វាអនុញ្ញាតឱ្យ Server ផ្ញើ Executable Code ទៅ Client (ជាធម្មតា JavaScript) ដើម្បីពង្រីក Functionality របស់ Client ។ ទោះបីជាមិនត្រូវការសម្រាប់ API ភាគច្រើន ក៏ដោយ វាផ្តល់ភាពបត់បែនបន្ថែម។

ការប្រើប្រាស់ Code on Demand

Browser Download JavaScript: Web Browsers ទាញយក JavaScript Scripts ពី Server ដែលជាឧទាហរណ៍ល្អបំផុតនៃ Code on Demand ។ Scripts ទាំងនេះ Execute ស្ទើរ Client-side ដោយពង្រីក Capability របស់ Browser។

ការពិចារណា Security

ការប្រើ Code on Demand ត្រូវការការពិចារណា Security យ៉ាងប្រុងប្រយ័ត្ន ពីព្រោះ Executable Code ដែល Download ពី Remote Server អាចមាន Security Risks ។ ត្រូវប្រាកដថាកូដដែលប្រគល់ទៅ Client ត្រូវបាន Validate ជាដំបូង។

៣. គោលការណ៍រចនា REST API

ក្នុងការបង្កើត REST-based API មួយ មានជាច្រើនជំហានត្រូវប្រព្រឹត្តទៅ មុនពេលយើងចាប់ផ្តើមសរសេរកូដ។ ក្រុមអ្នកអភិវឌ្ឍន៍ត្រូវតែទាក់ទងសន្ទនាជាមួយអ្នកពាក់ព័ន្ធ និងវិភាគតម្រូវការជាមុនសិន។ បន្ទាប់មក ពួកគេត្រូវតែកំណត់ User Stories (ឬ Job Stories) ដើម្បីកំណត់លទ្ធផលដែលចង់បាន។

ដំណើរការរចនា API ត្រឹមត្រូវ ក្នុងការរចនា REST API ដ៏ល្អ ក្រុមការងារត្រូវឆ្លងកាត់ជំហានជាច្រើន ដើម្បីធ្វើឱ្យ API មានភាពសមហេតុផល, ស្ថិតស្ថេរ, ងាយប្រើ, និងអាច Scale បាន។ ការរំលងជំហានណាមួយ អាចបណ្តាលឱ្យ API ខ្សោយ ដែលពិបាក Maintain ក្នុងរយៈពេលវែង។

៣. គោលការណ៍រចនា REST API – ជំហានសំខាន់ៗ

01

កំណត់ធនធាន API

Identify the resources – កំណត់ Objects ឬ Entities ដែល API ត្រូវគ្រប់គ្រង

02

កំណត់ទំនាក់ទំនង

កំណត់ទំនាក់ទំនងរវាង Resource នីមួយៗ ដូចជា One-to-Many, Many-to-Many

03

កំណត់ប្រតិបត្តិការ

កំណត់ព្រឹត្តិការណ៍ប្រតិបត្តិការ – Actions ដែល API ត្រូវអនុញ្ញាតឱ្យ Client ធ្វើ

04

រចនា URL Paths

ការរចនា URL Paths សម្រាប់ Resources ក្នុង REST API ឱ្យ Consistent

05

ផ្គូផ្គង HTTP Verbs

ផ្គូផ្គងប្រតិបត្តិការ API ជាមួយ HTTP Verb ត្រឹមត្រូវ (GET, POST, PUT, DELETE)

06

Response Codes & Documentation

ការផ្តល់កូដឆ្លើយតប និងការកំណត់ឯកសារ API ឱ្យល្អ

៣.១. កំណត់ធនធាន API

ជំហានដំបូងក្នុងការរចនា REST API គឺការកំណត់ Resources ។ Resource គឺជា Object ឬ Concept ណាមួយដែលអ្នកចង់ Expose ជាមួយ API ដូចជា User, Product, Order, Article ជាដើម។ Resources ភាគច្រើនទំនើបឈ្មោះ Nouns (ពាក្យនាម) ហើយ Plural ។

ប្រើ Nouns, មិនមែន Verbs

URL ត្រូវតំណាងឲ្យ Resource ដូចជា `/products` ហើយ Action ត្រូវបំណែងដោយ HTTP Method

ប្រើ Plural

ជាទូទៅប្រើ Plural Nouns ដូចជា `/users`, `/orders`, `/articles` ដើម្បីបង្ហាញ Collections

Hierarchical Structure

ប្រើ Sub-resources ដើម្បីបង្ហាញ Relationships ដូចជា `/users/1/orders` ។

៣.២. កំណត់ទំនាក់ទំនងរវាង Resource នីមួយៗ

បន្ទាប់ពីកំណត់ Resources រួច ជំហានបន្ទាប់គឺការយល់ពីទំនាក់ទំនងរបស់ Resources ជាមួយគ្នា។ ទំនាក់ទំនងទាំងនេះ ហៅថា Relationships គឺជាអ្វីដែលកំណត់ Structure នៃ URL Path នៅក្នុង REST API របស់អ្នក។ ការយល់ច្បាស់អំពី Relationships ជួយអ្នករៀបចំ Endpoints ឱ្យ Logical និង Intuitive។

One-to-One

ទំនាក់ទំនង One-to-One ដូចជា User មាន Profile តែមួយ។ ឧទាហរណ៍: `/users/1/profile`

One-to-Many

ទំនាក់ទំនង One-to-Many ដូចជា User មាន Orders ច្រើន។ ឧទាហរណ៍: `/users/1/orders`

Many-to-Many

ទំនាក់ទំនង Many-to-Many ដូចជា Products មាន Tags ច្រើន។ ឧទាហរណ៍: `/products/5/tags`

៣.៣. កំណត់ព្រឹត្តិការណ៍ប្រតិបត្តិការ

ព្រឹត្តិការណ៍ប្រតិបត្តិការ (Operations) គឺជា Actions ទាំងអស់ដែល API របស់អ្នកត្រូវ Support ។ Actions ទាំងនេះត្រូវបានផ្គូផ្គងជាមួយ HTTP Methods ដូចជា GET, POST, PUT, PATCH, DELETE ។ ការកំណត់ Operations ជាក់លាក់ជួយអ្នករៀបចំ Endpoints ឱ្យ Complete និង Consistent ។

List (ទាញយកទាំងអស់)

ទាញយក Collection ទាំងអស់នៃ Resource ដូចជា Users, Products ជាដើម

Get (ទាញយកតែមួយ)

ទាញយក Resource មួយ ដោយប្រើ Identifier ជាក់លាក់

Create (បង្កើត)

បង្កើត Resource ថ្មីក្នុង Collection

Update / Delete

កែប្រែ ឬ លុប Resource ដែលមានដោយប្រើ Identifier

៣.៤. ការរចនា URL Paths

ការរចនា URL Paths ល្អ គឺជាផ្នែកមួយដ៏សំខាន់ក្នុងការបង្កើត REST API ។ URLs ត្រូវតែ Consistent, Predictable, និងបង្ហាញ Resource Hierarchy ឱ្យ Logical ។ URL ល្អ ជួយឱ្យ Developer ផ្សេងៗ ងាយ Understand API ដោយមិនចាំបាច់ Token ឯកសារ Documentation ច្រើន។

Collections

`GET /api/products` – ទាញយក Products ទាំងអស់

`POST /api/products` – បង្កើត Product ថ្មី

Individual Resource

`GET /api/products/5` – ទាញយក Product ID=5

`PUT /api/products/5` – Update Product ID=5

`DELETE /api/products/5` – លុប Product ID=5

Nested Resources

`GET /api/users/1/orders` – ទាញយក Orders របស់ User ID=1

`GET /api/users/1/orders/7` – ទាញយក Order ជាក់លាក់

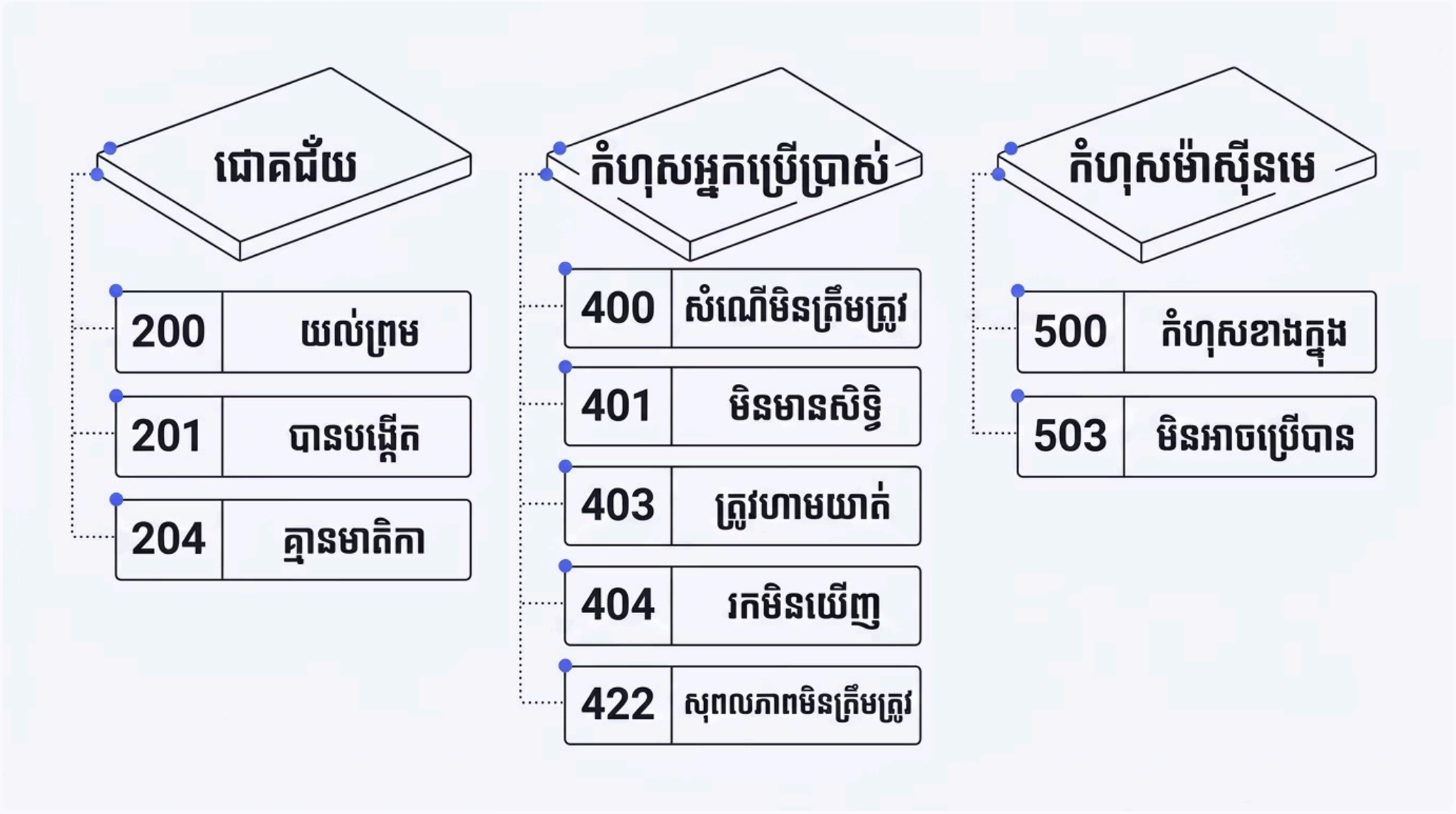
៣.៥. ផ្គូផ្គងប្រតិបត្តិការ API ជាមួយ HTTP Verb

ក្នុង REST API HTTP Methods (ឬ Verbs) ត្រូវបានប្រើដើម្បីបង្ហាញ Action ដែល Client ចង់ធ្វើលើ Resource ។ ការប្រើ HTTP Methods ត្រឹមត្រូវ ធ្វើឱ្យ API Predictable, Semantic, និង Consistent សម្រាប់ Developer ទាំងអស់។

HTTP Method	ប្រតិបត្តិការ	ឧទាហរណ៍
GET	Read / ទាញយក	GET /api/products – ទាញយក Products ទាំងអស់
POST	Create / បង្កើត	POST /api/products – បង្កើត Product ថ្មី
PUT	Update (ទាំងអស់)	PUT /api/products/5 – Update Product ID=5
PATCH	Update (ផ្នែក)	PATCH /api/products/5 – Update ជ្រើសរើស Fields
DELETE	Delete / លុប	DELETE /api/products/5 – លុប Product ID=5

៣.៦. ការផ្តល់កូដឆ្លើយតប (Response Codes)

HTTP Status Codes ជួយ Client ដឹងថា Request ជោគជ័យ ឬ បរាជ័យ ហើយ ហេតុអ្វី។ ការប្រើ Status Codes ត្រឹមត្រូវ ធ្វើឱ្យ API Transparent, Predictable, ហើយ Developer ងាយ Debug។



៣.៧. ការកំណត់ឯកសារ (Documentation) សម្រាប់ API

ការកំណត់ Documentation ល្អ គឺជាផ្នែកមួយសំខាន់ក្នុងការបង្កើត API ។ Documentation ល្អ ជួយ Developer ផ្សេងៗ ងាយ Understand, Integrate, និង Test API ។ ក្នុង ASP.NET Core, Swagger (OpenAPI) គឺជា Tool ដ៏ពេញនិយម ក្នុងការបង្កើត API Documentation ដោយស្វ័យប្រវត្តិ។

Swagger / OpenAPI

ក្នុង ASP.NET Core ត្រូវប្រើ `Swashbuckle.AspNetCore` Package ដើម្បីបង្កើត Swagger UI ដែល Auto-generate Documentation ពី Code ។

XML Comments

ប្រើ XML Documentation Comments (`///`) នៅក្នុង Controller Actions ដើម្បីផ្តល់ Description, Parameters, Response ឱ្យ Swagger ។

Interactive Testing

Swagger UI អនុញ្ញាតឱ្យ Developer ធ្វើ Test Endpoints ផ្ទាល់ក្នុង Browser ដោយមិនចាំបាច់ប្រើ Tool ខាងក្រៅ ។

៤. សេចក្តីណែនាំអំពី CRUD

CRUD គឺជាពាក្យកាត់សម្រាប់ បង្កើត (Create), អាន/យកទិន្នន័យ (Read), កែប្រែ (Update), និងលុប (Delete)។ វាមកពីប្រព័ន្ធ database ប្រភេទ relational ហើយតំណាងឲ្យប្រតិបត្តិការគ្រឹះ ៤ យ៉ាងសម្រាប់ការផ្ទុក និងគ្រប់គ្រងទិន្នន័យនៅក្នុងឃ្លាំងទិន្នន័យ។

Create

បង្កើតទិន្នន័យថ្មីចូលក្នុង Database ប្រើ HTTP POST

Read

ទាញយក / អានទិន្នន័យពី Database ប្រើ HTTP GET

Update

កែប្រែ / ធ្វើបច្ចុប្បន្នភាពទិន្នន័យ ប្រើ HTTP PUT ឬ PATCH

Delete

លុបទិន្នន័យចេញពី Database ប្រើ HTTP DELETE

៤.១. ការផ្គូផ្គងប្រភេទសំណើ (Request Types) ជាមួយប្រតិបត្តិការក្នុង Database

	Endpoint operation	Database operation
Create	POST request	INSERT
Read	GET request	SELECT
Update	PUT request	UPDATE
Delete	DELETE request	DELETE

រូបភាពខាងលើបង្ហាញពីការផ្គូផ្គង HTTP Request Types ជាមួយប្រតិបត្តិការ CRUD ក្នុង Database ។ ការផ្គូផ្គងទាំងនេះ ជួយ Developer ដឹងច្បាស់ថា ត្រូវប្រើ HTTP Method ណា ក្នុងស្ថានភាពណា ដើម្បីបង្កើត API ដែលស្របតាម REST Convention ។

៤.២. Endpoints ទាំងអស់ដែលត្រូវការ ដើម្បីគ្របដណ្តប់ប្រតិបត្តិការ CRUD

ដើម្បីគ្របដណ្តប់ CRUD Operations ទាំងស្រុង ក្នុង REST API មួយ ជាធម្មតាយើងបង្កើត Endpoints ខាងក្រោម ។ Endpoints ទាំងនេះ ជាស្តង់ដារ ហើយ Developer ភាគច្រើនស្គាល់ Pattern នេះ ដូច្នេះ API ងាយ Integrate ។

HTTP Method	Endpoint	ការពិពណ៌នា
GET	/api/resources	ទាញយក Resources ទាំងអស់ (List)
GET	/api/resources/{id}	ទាញយក Resource តែមួយ ដោយ ID
POST	/api/resources	បង្កើត Resource ថ្មី
PUT	/api/resources/{id}	Update Resource ទាំងស្រុង ដោយ ID
PATCH	/api/resources/{id}	Update Resource ផ្នែកណាមួយ ដោយ ID
DELETE	/api/resources/{id}	លុប Resource ដោយ ID

៥. Implementing GET Request

ការស្នើសុំ GET គឺមួយក្នុងចំណោមអ្វីដែលងាយបំផុតក្នុងការអនុវត្ត ពីព្រោះអ្វីៗភាគច្រើនបានភ្ជាប់រួចហើយនៅក្នុងកម្មវិធី។ ក្នុង ASP.NET Core Web API, Controller Action ត្រូវបាន Decorate ជាមួយ `[HttpGet]` Attribute ។ Framework នឹង Route Requests ដោយស្វ័យប្រវត្តិទៅ Action Method ត្រឹមត្រូវ ដោយផ្អែកលើ URL Pattern និង HTTP Method ។

Controller Setup

ក្នុង ASP.NET Core ត្រូវវាយ Attribute `[ApiController]` និង `[Route("api/[controller]")]` លើ Class ដើម្បីកំណត់ Base Route ។

Action Method

Action Method ដែល Handle GET Requests ប្រើ `[HttpGet]` Attribute ហើយ Return Type ជា `IActionResult` ឬ `ActionResult<T>` ។

Return Types

ប្រើ `Ok(data)` ដើម្បី Return 200 OK Response ព្រមទាំង Data ។ ប្រើ `NotFound()` ប្រសិនបើ Resource មិនត្រូវបានរក ។

៥.១. ការអនុវត្តប្រតិបត្តិការ GET Request (ទាំងអស់)

ការអនុវត្ត **GET Request (ទាញយកទិន្នន័យទាំងអស់)** នៅក្នុង **ASP.NET Core Web API** គឺប្រើសម្រាប់ទាញយក Resource ទាំងអស់ពី Database ឬ Source មួយ។ ជាធម្មតា Endpoint នេះ Return List<T> ហើយប្រើ HTTP Status Code 200 OK ជា Response ។

```
[HttpGet]
public async Task<ActionResult<IEnumerable<ProductDto>>> GetAll()
{
    var products = await _productService.GetAllAsync();
    return Ok(products);
}
```

Async / Await

ប្រើ `async/await` Pattern ដើម្បីធ្វើ Database Calls ក្នុងលក្ខណៈ Non-blocking ធ្វើឱ្យ Application Scale ជាមួយ Traffic ច្រើន

DTO Pattern

Return DTO (Data Transfer Object) មិនមែន Database Entity ទាំងស្រុង ដើម្បីបំបែក Data ដែល Expose ទៅ Client

Status 200 OK

ប្រើ `Ok()` Wrapper ដើម្បី Return 200 Status Code ព្រមជាមួយ Data ក្នុង Response Body

៥.២. ការអនុវត្តប្រតិបត្តិការ GET Request By ID

នៅក្នុង RESTful API ប្រតិបត្តិការ GET By Id គឺសម្រាប់ ទាញយកធាតុ (Item) មួយតែប៉ុណ្ណោះ ដោយប្រើ Parameter {id} នៅក្នុង URL ។ Endpoint នេះ ត្រូវ Return Resource មួយ ឬ 404 Not Found ប្រសិនបើ Resource នោះ មិនមានក្នុង Database ។

```
[HttpGet("{id}")]
public async Task<ActionResult<ProductDto>> GetById(int id)
{
    var product = await _productService.GetByIdAsync(id);
    if (product == null)
        return NotFound();
    return Ok(product);
}
```

Route Parameter {id}

Parameter `{id}` ក្នុង `[HttpGet("{id}")]` ត្រូវ Match ជាមួយ Method Parameter `int id` ដោយស្វ័យប្រវត្តិ

Return 404 Not Found

ប្រសិនបើ Resource មិនត្រូវបានរក ត្រូវ Return `NotFound()` ដែល Produce HTTP 404 Status Code ។ នេះ Semantic ត្រឹមត្រូវ ហើយ Client ដឹងថា Resource មិនមាន

៥.៣. ការអនុវត្តប្រតិបត្តិការ GET Request By ID ជាមួយ Related Resource

ក្នុង REST API ច្រើនដង Resource មួយ មាន Related Resources ជាមួយ ។ ឧទាហរណ៍ User មាន Orders ច្រើន ។ ដើម្បីទាញយក Related Resource ជាក់លាក់ យើងប្រើ Nested Route Pattern ។ Endpoint នេះ ច្រើន Return Related Object ជា Part of Response ។

```
[HttpGet("{userId}/orders/{orderId}")]
public async Task<ActionResult<OrderDto>> GetUserOrder(
    int userId, int orderId)
{
    var user = await _userService.GetByIdAsync(userId);
    if (user == null) return NotFound("User not found");

    var order = await _orderService
        .GetByIdAndUserAsync(orderId, userId);
    if (order == null) return NotFound("Order not found");

    return Ok(order);
}
```

ចំណាំ: ក្នុង Nested Route ត្រូវវា Validate Parent Resource ជាមុន (User) ហើយទើបចូល Child Resource (Order) ។ ការ Validate នៅ Multiple Levels ជួយ Return Error Message ដែល Informative ជាង។

៦. ការអនុវត្តប្រតិបត្តិការ POST ក្នុង Web API

POST request គឺសម្រាប់ ✔ បង្កើតធាតុថ្មី (Create) ✔ ផ្ញើទិន្នន័យពី Client → Server ✔ រក្សាទុកក្នុង Database ។ ក្នុង ASP.NET Core Action Method ដែល Handle POST ប្រើ `[HttpPost]` Attribute ហើយ Accept Request Body ជា DTO Object ។

```
[HttpPost]
public async Task<ActionResult<ProductDto>> Create(
    [FromBody] CreateProductDto dto)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    var created = await _productService.CreateAsync(dto);
    return CreatedAtAction(
        nameof(GetById),
        new { id = created.Id },
        created);
}
```

[FromBody]

Attribute `[FromBody]` ប្រាប់ ASP.NET ឱ្យ Deserialize JSON Body ជា DTO Object ដោយស្វ័យប្រវត្តិ

201 Created

`CreatedAtAction()` Return HTTP 201 ព្រមជាមួយ Location Header ដែលចង្អុល URL របស់ Resource ថ្មី

Validation

ប្រើ `ModelState.IsValid` ដើម្បីពិនិត្យ Input ។ ប្រសិនបើ Invalid Return 400 Bad Request

៧. ការអនុវត្តប្រតិបត្តិការ PUT ក្នុង Web API

PUT ត្រូវបានប្រើសម្រាប់ ✔ កែប្រែ (update) ទិន្នន័យដែលមានរួចនៅក្នុង database ✔ ជំនួស Resource ចាស់ដោយ Resource ថ្មី ✔ ត្រូវមាន Id ក្នុង URL ។ PUT ជំនួស Resource ទាំងស្រុង ខុសពី PATCH ដែល Update ផ្នែកតែ ។

```
[HttpPut("{id}")]
public async Task<IActionResult> Update(
    int id, [FromBody] UpdateProductDto dto)
{
    if (id != dto.Id)
        return BadRequest("ID mismatch");

    var exists = await _productService.ExistsAsync(id);
    if (!exists) return NotFound();

    await _productService.UpdateAsync(dto);
    return NoContent();
}
```

204 No Content

PUT ជោគជ័យ ជាធម្មតា Return 204 No Content (ជោគជ័យ ប៉ុន្តែ គ្មាន Body ត្រឡប់) ។ ខ្លះ Return 200 OK ជា Alternative

ID Mismatch Check

ត្រូវវា Validate ថា ID ក្នុង URL ត្រូវ Match ជាមួយ ID ក្នុង Body ដើម្បី Prevent Inconsistency ។ ប្រសិនបើ Mismatch Return 400 Bad Request

៨. ការអនុវត្តប្រតិបត្តិការ HEAD ក្នុង Web API

HEAD Request គឺជាប្រភេទ Request ក្នុង HTTP ដែលទាក់ទងនឹង GET ប៉ុន្តែ ➡️ វាមិនបញ្ជូន body data ត្រឡប់មកវិញទេ ➡️ វាបញ្ជូនតែ headers ប៉ុណ្ណោះ ។ HEAD Request ប្រើជាញឹកញាប់ ដើម្បី Check ថា Resource មានក្នុង Server ដោយមិន Download Content ទាំងស្រុង ដែលសន្សំ Bandwidth ។

ពិនិត្យ Resource Existence

HEAD ប្រើ Check ថា Resource មានក្នុង Server ។ ប្រសិនបើ Resource មាន Server Return 200 OK ។ ប្រសិនបើ Resource មិនមាន Return 404 Not Found ។ គ្មាន Body ។

ទទួល Metadata

HEAD ប្រើ Get Headers ដូចជា Content-Length, Content-Type, Last-Modified ដោយ Download Content ពិតប្រាកដ ដូច្នេះ Client ដឹងអំពី Resource ដោយ Efficient

សន្សំ Bandwidth

ក្នុង ASP.NET Core HEAD Requests ជាធម្មតាត្រូវ Handled ដោយ GET Action ដូចគ្នា Framework Remove Body ដោយស្វ័យប្រវត្តិ ។

៩. ការអនុវត្តប្រតិបត្តិការ DELETE ក្នុង Web API

DELETE Request ត្រូវបានប្រើសម្រាប់ ✔ លុប (remove) resource មួយចេញពី database ✔ ប្រើ Id ក្នុង URL ✔ Return status code បង្ហាញភាពជោគជ័យ ឬបរាជ័យ ។ DELETE Operation ភ្ជាក់ប្រឆាំងថ្លែ ពីព្រោះ Data ដែលត្រូវបានលុប ជាទូទៅ មិនអាច Recover បានទៀតទេ ។

```
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var exists = await _productService.ExistsAsync(id);
    if (!exists) return NotFound();

    await _productService.DeleteAsync(id);
    return NoContent();
}
```

204 No Content

DELETE ជោគជ័យ ជាធម្មតា Return 204 No Content ។ Resource ត្រូវបានលុប ហើយ Body ត្រឡប់ Empty

404 Not Found

ប្រសិនបើ Resource ដែលចង់លុបមិនមាន Return 404 Not Found ដើម្បីជូន Client ដឹង

Soft Delete Pattern

API ច្រើន Implement Soft Delete (Mark as Deleted) ជំនួស Hard Delete ដើម្បី Preserve Data History ។ ក្នុងករណីនេះ Return 200 OK

១០. គន្លឹះល្អបំផុតសម្រាប់រចនានិងសរសេរ API

ការរចនា API ល្អជាអ្វីដែលធ្វើអោយសេវាកម្មមានភាពងាយប្រើ, មានសុវត្ថិភាព, មានល្បឿនលឿន និងងាយសំរាប់ maintenance និង scaling។ ខាងក្រោមនេះជាគន្លឹះសំខាន់ៗដែលអ្នកគួរតែអនុវត្តនៅពេលបង្កើត ASP.NET Core Web API ។ គ្រប់ Best Practice ទាំងនេះ ត្រូវបានប្រើប្រាស់ ដោយ Industry Leaders ហើយ Proven ថា Improve Quality, Developer Experience, និង System Reliability ។

១០.១. Use Resource-Based URLs

គោលការណ៍ Resource-Based URLs មានន័យថា URL ត្រូវតំណាងឲ្យ Resource (Noun) ហើយ HTTP Method ទើបតំណាងឲ្យ Action (Verb) ។ ការប្រើ Verbs ក្នុង URL ដូចជា `/getUser` ឬ `/deleteProduct` ចាត់ទុកថាផ្ទុយពី REST Convention ។

✅ ត្រឹមត្រូវ (RESTful)

`GET /api/users` – ទាញយក Users ទាំងអស់

`POST /api/users` – បង្កើត User ថ្មី

`GET /api/users/5` – ទាញយក User ID=5

`DELETE /api/users/5` – លុប User ID=5

❌ ខុស (Non-RESTful)

`GET /api/getUsers`

`POST /api/createUser`

`GET /api/getUserById?id=5`

`POST /api/deleteUser/5`

១០.២. Use Standard HTTP Methods Correctly

ប្រើ HTTP Methods ត្រឹមត្រូវ (Use Standard HTTP Methods Correctly) ។ HTTP Methods នីមួយៗ មាន Semantic ជាក់លាក់ ហើយ Developer ច្រើនពឹងផ្អែកលើ Semantics ទាំងនោះ ដើម្បី Predict Behavior នៃ API ។ ការប្រើ Methods ខុស ជំរុញ Confusion, Bugs, និង Security Vulnerabilities ។

Method	Safe / Idempotent?	ការប្រើ
GET	Safe + Idempotent	ទាញយកទិន្នន័យ – ជានិច្ចមិន Modify Data
POST	Neither	បង្កើតថ្មី – Response ខុសគ្នារាល់ Request
PUT	Idempotent	Replace Resource ទាំងស្រុង – Result ដូចគ្នា
PATCH	Not guaranteed	Update ផ្នែកណាមួយ – ជំនួស Fields ជ្រើស
DELETE	Idempotent	លុប Resource – Result ដូចគ្នា ប្រសិនបើ Call ម្តងទៀត

១០.៣. Use Correct HTTP Status Codes

ប្រើ Response Codes ត្រឹមត្រូវ (Use Correct HTTP Status Codes) ។ HTTP Status Codes ដើម្បីឱ្យ Client ដឹងថា Request ជោគជ័យ ឬ បរាជ័យ ហើយ ហេតុអ្វី ។ ការ Return 200 OK ក្នុងគ្រប់ Situation ជ្រើស ធ្វើឱ្យ Client ពិបាក Handle Errors ។

2xx – ជោគជ័យ

200 OK, 201 Created, 204 No Content – ប្រើត្រូវ Context ។ 201 ជ POST, 204 ជ PUT/DELETE

4xx – Client Error

400 Bad Request (Invalid Input), 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable Entity (Validation Error)

5xx – Server Error

500 Internal Server Error – ដើម្បី Log ហើយ Return Generic Error Message ទៅ Client ។ មិន Expose Stack Trace ទៅ Production

១០.៤. Support Pagination, Filtering, Sorting

គាំទ្រ Pagination និង Filter (Support Pagination, Filtering, Sorting) ។ API ដែល Return Data ច្រើន ត្រូវរា Support Pagination, Filtering, Sorting ដើម្បី Prevent ការ Load Data ច្រើនពេក ដែលធ្វើឱ្យ Performance ខ្សោយ និង Client ពិបាក Handle ។

```
// Query Parameters: /api/products?page=1&pageSize=10&sort=name&filter=active
[HttpGet]
public async Task<ActionResult> GetAll(
 [FromQuery] int page = 1,
 [FromQuery] int pageSize = 10,
 [FromQuery] string sort = "id",
 [FromQuery] string filter = null)
{
 var result = await _productService
 .GetPagedAsync(page, pageSize, sort, filter);

 Response.Headers.Add("X-Total-Count",
 result.TotalCount.ToString());
 return Ok(result.Items);
}
```

១០.៥. Use DTO Models

ប្រើ DTO ឲ្យបានត្រឹមត្រូវ (Use DTO Models) ។ DTO (Data Transfer Object) ជា Object ពិសេសដែលប្រើដើម្បី Transfer Data រវាង Layers ។ ជំនួសនឹង Expose Database Entities ផ្ទាល់ ត្រូវប្រើ DTOs ដើម្បី Control ថាតើ Fields ណាដែល Expose ទៅ Client, ការពារ Over-posting, និងធ្វើឱ្យ API Contract ខុសពី Internal Data Structure ។

ការពារ Sensitive Data

DTO ជួយ Prevent ការ Expose Sensitive Fields (ដូចជា Password Hash, Internal IDs) ទៅ Client ។ ប្រើ `ProductDto` (Expose) ជំនួស `Product` Entity

ការពារ Over-posting

DTO ការពារ Over-posting Attack ដែល Client ដាក់ Fields ដែលមិន Allow ។ `CreateProductDto` មានតែ Fields ដែល Client អាច Set

AutoMapper

ប្រើ AutoMapper Library ដើម្បី Map ដោយស្វ័យប្រវត្តិ រវាង Entity ↔️ DTO ។ ជំនួសការ Manual Mapping ដែលបរិបូណ៌ Boilerplate Code

១០.៦. Validate All Inputs

Validate Request ទាំងអស់ (Validate All Inputs) ។ ការ Validate Input គឺជា Security Practice ដ៏ចាំបាច់ ។ Input ដែល Invalid អាចបណ្តាល ឱ្យ Business Logic Errors, Data Corruption, ឬ Security Vulnerabilities ។ ក្នុង ASP.NET Core ប្រើ Data Annotations ឬ FluentValidation Library ។

```
public class CreateProductDto
{
    [Required(ErrorMessage = "Name is required")]
    [StringLength(100, MinimumLength = 2)]
    public string Name { get; set; }

    [Range(0.01, double.MaxValue,
        ErrorMessage = "Price must be positive")]
    public decimal Price { get; set; }

    [Required]
    [MaxLength(500)]
    public string Description { get; set; }
}
```

ASP.NET Core Tip: ក្នុង Controller ដែលប្រើ `[ApiController]` Attribute, Model Validation ដំណើរការ Automatically ។ ប្រសិនបើ Model Invalid, Framework Return 400 Bad Request ដោយ Automatic ដោយមិនចាំបាច់ Check `ModelState.IsValid` ។

១០.៧. API Versioning

API Versioning ជួយ Maintain Backward Compatibility នៅពេលដែលអ្នកត្រូវវា Update ឬ Change API ។ ដោយ Version API, Client ដែលប្រើ Version ចាស់ ស្ថិតស្ថេរ ហើយ Client ថ្មីអាចបានយ៉ាង Benefit ពី Features ថ្មី ។

URL Path Versioning

រួមមាន Version ក្នុង URL Path: `/api/v1/products`, `/api/v2/products` ។ ជា Strategy ល្អ និង ងាយ Understand

Header Versioning

ដាក់ Version ក្នុង Request Header: `X-API-Version: 2` ។ URL Clean ប៉ុន្តែ ពិបាក Test ជាង ពីព្រោះ Header Hidden

Query String Versioning

ដាក់ Version ជា Query Parameter: `/api/products?version=2` ។ ងាយ Test ប៉ុន្តែ URL ច្រំ

```
// Install: Microsoft.AspNetCore.Mvc.Versioning
builder.Services.AddApiVersioning(options => {
 options.DefaultApiVersion = new ApiVersion(1, 0);
 options.AssumeDefaultVersionWhenUnspecified = true;
 options.ReportApiVersions = true;
});
```

១០.៨. Swagger Documentation

Swagger (OpenAPI) គឺជា Standard ដ៏ពេញនិយម ក្នុងការ Document API ។ ក្នុង ASP.NET Core ប្រើ `Swashbuckle.AspNetCore` Package ដើម្បី Auto-generate Swagger UI ពី Code ។ Developer ផ្សេងៗ អាចទៅ `/swagger` Endpoint ហើយ Test API ផ្ទាល់ ដោយ Interactive ។

```
// Program.cs
builder.Services.AddSwaggerGen(c => {
    c.SwaggerDoc("v1", new OpenApiInfo {
        Title = "My API",
        Version = "v1",
        Description = "ASP.NET Core Web API"
    });
    // Include XML Comments
    var xmlFile = $"{Assembly.GetName().Name}.xml";
    var xmlPath = Path.Combine(AppContext.BaseDirectory, xmlFile);
    c.IncludeXmlComments(xmlPath);
});

app.UseSwagger();
app.UseSwaggerUI();
```

XML Comments ក្នុង Actions

ប្រើ XML Comments (`///`) លើ Action Methods ដើម្បីផ្តល់ Description, Parameters, Response Codes ទៅ Swagger Documentation ។

Production Setup

ក្នុង Production Environment ត្រូវរា Secure ឬ Disable Swagger UI ។ Swagger ត្រូវ Enable ក្នុង Development/Staging Environment ប៉ុណ្ណោះ ដើម្បី Protect API Information។

១០.៩. Security (JWT + HTTPS)

Security (JWT + HTTPS) ជាផ្នែកចាំបាច់ក្នុង Production API ។ JWT (JSON Web Tokens) ប្រើសម្រាប់ Authentication, Authorization ។ HTTPS ធ្វើ Encrypt Traffic រវាង Client និង Server ។ ដោយគ្មាន HTTPS, JWT Token អាច Intercepted ហើយ Compromised ។

JWT Authentication

JWT Token ចាប់ Signed ដោយ Secret Key ។ Server Verify Signature ដើម្បីបញ្ជាក់ Token Authenticity ។ Token ផ្ទុក Claims (User ID, Role, Expiry) ហើយ Stateless

HTTPS Enforcement

ប្រើ `app.UseHttpsRedirection()` ដើម្បី Force Redirect HTTP → HTTPS ។ ក្នុង Production ទទួល SSL Certificate (Free via Let's Encrypt)

Authorization Policies

ប្រើ `[Authorize]` Attribute, Role-Based Authorization, Policy-Based Authorization ដើម្បី Control Access ទៅ API Endpoints ជាក់លាក់

១០.១០. Caching

Caching ជា Best Practice ដ៏សំខាន់ ដើម្បីធ្វើ Performance ឱ្យ API ប្រសើរ ។ ការ Cache Responses កាត់បន្ថយ Database Load, ។ ក្នុង ASP.NET Core មាន Built-in Memory Cache ហើយ Support Redis Distributed Cache សម្រាប់ Multi-Server Scenarios ។

In-Memory Cache

ក្នុង Single Server ប្រើ `IMemoryCache` ។ Simple, Fast ប៉ុន្ត Limited ទៅ RAM នៃ Server ។ ល្អ Development, Small Scale

Distributed Cache (Redis)

ក្នុង Multi-Server ប្រើ Redis (LabProvider) ។ Cache Shared រវាង Servers ។ ល្អ Production, High Availability

Response Caching

ប្រើ `[ResponseCache]` Attribute ដើម្បី Add Cache Headers ទៅ HTTP Response ។ CDN, Proxy ្ត Cache Responses ។

១០.១១. Consistent Naming, Asynchronous Operations

Consistent Naming, Asynchronous Operations ជាគន្លឹះ ដែល Combine ជាច្រើន Best Practices ។ Consistent Naming ធ្វើ API ងាយ Predict ។ Async Operations ធ្វើ API Scale ជាមួយ Load ច្រើន ។

Consistent Naming Convention

ប្រើ camelCase ក្នុង JSON Response, PascalCase ក្នុង C# Code ។ ប្រើ Plural Nouns ក្នុង URL ។ ប្រើ Suffix Async ក្នុង Async Method Names ។ Consistent ជួយ Developer ទស្សន៍ API ដោយ Easy

Async / Await Pattern

ប្រើ `async Task<T>` ក្នុង Controller Actions, Service Methods, Repository Methods ។ Async Calls ដល់ Database, External APIs ។ Non-blocking Operations ធ្វើ Thread Pool Free ដើម្បី Handle Requests ច្រើន

→ ប្រើ Async ជា Default

Method ណាក៏ដោយ ដែល Call Database, File System, ឬ Network ត្រូវ Async ។

→ Avoid .Result / .Wait()

ហាមមិនឱ្យ Block Async Code ដោយ `.Result` ឬ `.Wait()` ពីព្រោះ Cause Deadlocks

→ Naming ឱ្យ Consistent

Method Names, Property Names, URL Paths ត្រូវ Consistent ទូទាំង Codebase ។

១០. សរុបគន្លឹះល្អបំផុតសម្រាប់រចនានិងសរសេរ API

ខាងក្រោមនេះជាការ Summary នៃ Best Practices ទាំងអស់សម្រាប់ ASP.NET Core Web API ។ ការ Apply Best Practices ទាំងនេះ ជួយ Ensure ថា API របស់អ្នក Professional, Maintainable, Secure, និង Scalable ។

Swagger Documentation

Auto-generate API Docs ជា Interactive Swagger UI ដើម្បី Developer ងាយ Test

Security (JWT + HTTPS)

Protect API ជាមួយ JWT Authentication, Authorization Policies, HTTPS Enforcement

Pagination & Filtering

Support Page, Sort, Filter Query Parameters ដើម្បី Prevent Data Overload

Caching

In-Memory Cache ឬ Redis ដើម្បី Reduce Database Calls, Improve Speed

Consistent Naming

camelCase JSON, PascalCase C#, Plural Nouns URLs, Async Suffix

Asynchronous Operations

Async/Await ក្នុង IO Operations ទាំងអស់ ដើម្បី Scale ជាមួយ High Traffic